

DNS Tunnelling Detection Tool

MAX FERSTENBERG

5/13/26

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	1
1.3	Aims	1
1.4	Scope	1
1.5	Report Structure	1
2	Background	1
2.1	Detection Methods	1
2.2	Frequency Analysis.....	2
2.3	Tunnelling Tools	2
3	Methodology	2
3.1	System Architecture	2
3.2	Lab Environment	2
3.3	Data Generation	3
3.4	Parsing.....	3
3.5	Feature Extraction	3
3.6	Baseline Construction	3
3.7	FFT Beaconsing Detection	3
3.8	Classification	4
3.9	Evaluation	4
4	Implementation	4
4.1	Project Structure	4
4.2	Lab Environment	5
4.3	Parser Module	7
4.4	Baseline Construction	8
4.5	Feature Extraction Module	9
4.6	FFT Module.....	10
4.7	Classifier Module	11
4.8	Live Capture	14
4.9	Evaluation	16
5	Testing and Validation	19
5.1	Testing Methodology.....	19
5.2	Test Data.....	19
5.3	Module Testing	20

5.3.1	Parser Testing.....	20
5.3.2	Feature Extraction Tests.....	20
5.3.3	FFT Tests	22
5.3.4	Classifier Tests	22
5.3.5	Live Testing.....	24
5.4	Threshold Tuning	25
5.4.1	Feature Extraction Tuning.....	25
5.4.2	Normalisation Range	27
5.4.3	Minimum Features Required	28
6	Evaluation	29
7	Discussion	30
7.1	Implementation.....	30
7.2	Tools.....	31
7.3	Results.....	32
8	Conclusion.....	32
	References	34
	Appendices	37
	Appendix A: Requirements Breakdown	37
A.1	Functional Requirements.....	37
A.2	Non-Functional Requirements	37

1 Introduction

1.1 Motivation

Domain Name System (DNS) is a fundamental internet protocol that translates domain names into IP addresses; the ubiquitousness of DNS means it is rarely blocked on a network perimeter by organisations (Wang et al., 2021)^[1]. The inherent level of trust in DNS makes it an attractive vector for attackers; DNS tunnelling encodes data inside DNS queries and responses, allowing attackers to bypass conventional measures in a manner that is often tricky to monitor, given that it uses a legitimate protocol on a legitimate port (Salat, Davis and Khan, 2023)^[2].

1.2 Problem Statement

Existing methods are typically either signature-based, or Machine Learning (ML)-based. The former can catch known tools but misses novel variants (Wang et al., 2021)^[1], and the latter requires sanitised data, is computationally expensive and can't always explain its behaviour (Das et al., 2017)^[3].

1.3 Aims

Given the above problem statement (§1.2), this project aims to provide a middle-ground; a transparent and lightweight detection approach based on statistical consistencies in tunnelled traffic. Thus, the following aims are proposed to meet this goal. A full list of requirements is available in [Appendix A](#).

- Parse common DNS data formats.
- Extract traffic features for statistical analysis.
- Apply Kullback-Leibler (KL) Divergence and n-gram frequency analysis to analyse the extracted traffic features.
- Apply Fast Fourier Transforms (FFT) to analyse the time frequency to detect automated behaviour.

1.4 Scope

Based on the proposed problem statement (§1.2) and defined aims (§1.3), the project will be bounded by the following scope: Test data will be generated by using open-source tunnelling tools in a Docker environment; there will be no ML classification used; other categories of DNS abuse are out of scope.

1.5 Report Structure

§2 will provide a brief review on current literature and key concepts of DNS tunnelling. §3 sets out the proposed methodology for the tool. §4 details the implementation of the proposed methodology. §5 provides a log of the testing and validation process. §6 presents the evaluation of this process. §7 discusses findings, limitations and future work, all concluded in §8.

2 Background

2.1 Detection Methods

Detection broadly divides into payload analysis and traffic analysis. Payload analysis involves inspecting characteristics such as subdomain length and entropy per query, whereas traffic analysis aims to aggregate behaviour like query volume and record type distribution (Wang et al., 2021)^[1].

Shannon entropy quantifies the randomness of subdomain characters (Shannon and Weaver, 1949)^[4]; legitimate domains have character distributions like natural language while encoded payloads do not (Born and Gustafson, 2010)^[5].

These heuristics provide a more traceable classification technique without sanitised data, per §1.2.

2.2 Frequency Analysis

Tunnelling tools commonly maintain channels of communication through regular queries, also known as “beaconing”. FFT converts a temporal series into a frequency, to reveal distinguishing patterns between automated beaconing and human traffic; this approach is well-established for general command-and-control (C2) detection but relatively underexplored as applied specifically to DNS tunnel detection (Kim et al., 2016)^[6].

2.3 Tunnelling Tools

Iodine and dns2tcp are open-source tunnelling tools with distinct traffic signatures; iodine establishes a full IP tunnel using primarily NULL records, whereas dns2tcp forwards TCP connections using TXT records (Aiello, Merlo and Papaleo, 2012; Salat, Davis and Khan, 2023)^{[7][2]}.

3 Methodology

3.1 System Architecture

The system will be modelled as a pipeline with four distinct stages: parser, feature extractor, FFT analyser, classifier. The pipeline will operate on both static PCAP files or live traffic.

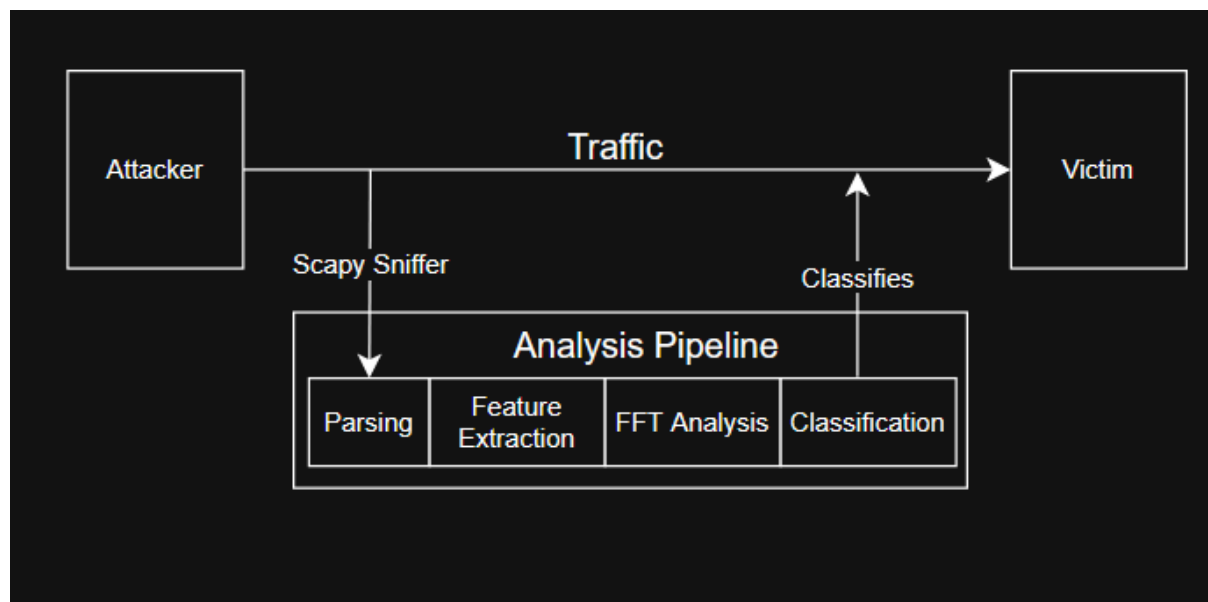


Figure 3.1: High-Level Architecture

3.2 Lab Environment

Two Docker containers (attacker and victim) will exist on a bridge network, isolated from the host VM’s DNS. Containers are built from a Debian image. Each container will have an IP address statically assigned for the purposes of explainability of analysis when it comes to

testing and evaluation. Iodine requires NET_ADMIN and TUN device interface; this will be included in the Docker compose configuration (Docker Documentation, 2024)^[8].

3.3 Data Generation

Data will be generated by benign browsing on the host VM, and tunnelled traffic via Iodine and dns2tcp. The tunnelling tools were chosen for the architectural differences discussed in §2.3, ensuring that the tool is tested across a variety of methods.

All capture is performed via *tcpdump* on the relevant network interface, saved as PCAP files. Live testing will be performed by running the same tunnelling tools while the detector sniffs the network bridge interface.

3.4 Parsing

The parser will read PCAP files via Scapy, filter out DNS packets, extract the timestamp, query name, record type and parent/subdomain split. Similarly, live sniffing will use Scapy's sniffing API on a specified network interface, producing records in the same format (Biondi, 2026)^[9].

Records will be output as a list of dictionaries. Latin-1 encoding will be used when decoding query names to preserve binary characters from Iodine.

3.5 Feature Extraction

When performing feature extraction, features are computed across all queries to that domain and aggregated per parent domain. The full list of extracted features is as follows (per §2.2/2.3):

- Shannon entropy
- KL divergence
- Bigram KL divergence
- Mean and standard deviation of subdomain length
- Query volume and rate
- Unique subdomain count and ratio
- Proportion of uncommon record types

Sample size guards will be put in place to ensure that features are computed only when statistically meaningful, so as to address bias in small samples with KL divergence and rate calculations. Features that cannot be reliably computed across a sample return None rather than a default value.

3.6 Baseline Construction

The Tranco top-domains list will be used as a baseline distribution of benign character and bigram frequencies; second-level domains (SLDs) will be extracted and counted over a fixed alphabet of lowercase letters, digits and hyphens (Le Pochat et al., 2019)^[10]. Characters outside the alphabet are bucketed into a miscellaneous category. The output static distribution is stored as a JSON file.

3.7 FFT Beaconing Detection

Query timestamps are resampled onto an evenly spaced grid with 1 second bins. FFT will be applied to this grid via NumPy, with the grid analysed for dominant frequency components (Harris et al., 2020)^[11]. The primary metric for detection of these dominant components will be the ratio of peak magnitude to median magnitude; domains with a peak to median ratio above a

defined threshold are flagged as beacons (Kim et al., 2016)^[6]. Output is then integrated as an additional feature in the classifier.

3.8 Classification

Each feature will be normalised into a range of 0 to 1. Composite score will be computed as a weighted average of available features. Specific weighting will be iteratively calibrated during testing and validation (§5) (Capuano et al., 2022)^[12]. The score threshold will produce a classification of either benign, suspicious, or tunnel. Missing features (returned as None per §3.5) will be excluded. The contributions of each feature will be retained the classification output for explainability per §1.2.

3.9 Evaluation

Classification will be evaluated against known labels from the data generation process in §3.3; all generated traffic will be known as tunnel or benign. Standard binary classification metrics will be computed, i.e., precision, recall and F1 score. Additionally, effectiveness will be analysed across tools to assess consistency and generalisation across encoding schemes and tunnelling methods. Lastly, false positive analysis will be performed on benign traffic to identify edge cases (Liu et al., 2017)^[13].

4 Implementation

4.1 Project Structure

The overarching repository follows a flat layout per module, with one directory per pipeline stage as proposed in §3.1: *parser/*, *features/*, *fft/*, *classifier/*, *detector/* (the live module). There are two supporting directories: *data/* and *baselines/* for the PCAP files and reference distribution respectively. Additionally, *lab/* contains the Dockerfile and compose for the environment proposed in §3.2.

Each pipeline stage is a self-contained module that can be tested and operated independently. All modules can be invoked from the CLI.

Configuration for external tools live as plaintext files mounted into containers; this is mostly applicable only to dns2tcp, but the discussed baseline distributions (§3.6) and classifier weights (§3.8) live as a JSON file and Python dictionary respectively.

Dependencies are documented in *requirements.txt*.

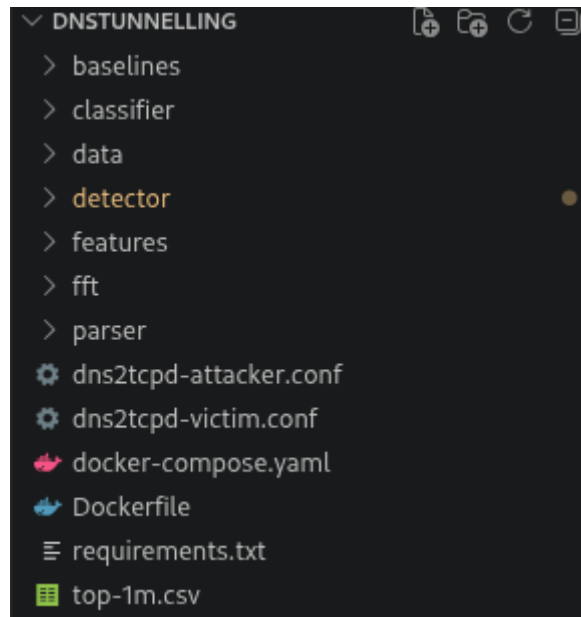


Figure 4.1A: Project Directory in VSCode

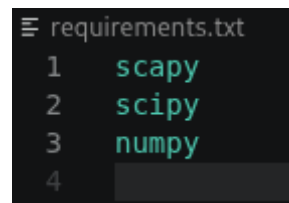


Figure 4.1B: requirements.txt

4.2 Lab Environment

There is a single Dockerfile based on *debian:bookworm-slim* for both containers. *Apt* installs are grouped into one *RUN* instruction with a cleanup invocation of */var/lib/apt/lists/** to minimise image size. Packages are selected as follows:

- *iputils-ping*, *bind9-dnsutils* for debugging.
- *git*, *build-essential*, *ruby*, *ruby-dev* to compile DNScat2.
- *iodine*, *dns2tcp* from *apt* per [§2.3](#)

For DNScat2, the client is then compiled with *make*, and the server is set up with *Bundler*.

```
Dockerfile > ...
1 FROM debian:bookworm-slim
2 WORKDIR /opt
3 RUN apt-get update && \
4     apt-get install -y iputils-ping bind9-dnsutils git build-essential iodine dns2tcp ruby ruby-dev && \
5     rm -rf /var/lib/apt/lists/* && \
6     git clone https://github.com/iagox86/dnscat2.git && \
7     cd /opt/dnscat2/client/ && \
8     make && \
9     cd /opt/dnscat2/server/ && \
10    gem install bundler && \
11    bundle install
```

Figure 4.2A: Dockerfile

The *docker-compose* file design is as follows (Docker Documentation, 2024)^[14]:


```

docker-compose.yaml
  > Run All Services
1  services:
   > Run Service
2      attacker:
3          build: .
4          container_name: dns-attacker
5          stdin_open: true
6          tty: true
7          cap_add:
8              - NET_ADMIN
9          devices:
10             - /dev/net/tun
11          networks:
12              lab-network:
13                  ipv4_address: 172.20.0.10
14          volumes:
15              - ./dns2tcpd-attacker.conf:/etc/dns2tcpd.conf
16
   > Run Service
17      victim:
18          build: .
19          container_name: dns-victim
20          stdin_open: true
21          tty: true
22          cap_add:
23              - NET_ADMIN
24          devices:
25              - /dev/net/tun
26          networks:
27              lab-network:
28                  ipv4_address: 172.20.0.20
29          volumes:
30              - ./dns2tcpd-victim.conf:/etc/dns2tcpd.conf

```

Figure 4.2B: docker-compose services

```

32  networks:
33      lab-network:
34          driver: bridge
35          ipam:
36              config:
37                  - subnet: 172.20.0.0/24

```

Figure 4.2C: docker-compose networks

Figure 4.2C shows the bridge network discussed in §3.2, named *lab-network* on subnet 172.20.0.0/24. Figure 4.2B shows the two containers running on this network with the static IP assignment; 172.20.0.10 and 172.20.0.20 for the attacker and victim respectively (Docker Documentation, 2024)^[15].

Additionally, both services have `NET_ADMIN` added to capabilities, and `/dev/net/tun` added to devices, as discussed for Iodine requirements in §3.2.

`stdin_open: true` and `tty: true` keep the containers alive. Volumes are used to mount the `dns2tcp` configs into the containers, as discussed in §4.1.

It should be noted that `NET_ADMIN` is a privileged setting, so the lab is intentionally isolated on a subnet with no routing or port forwarding to the host, and thus tunnelling tools cannot reach

the real internet from these containers (man7.org, n.d.)^[16]. Additionally, the *dns2tcp* configs are read-only.

```
(kali@kali)-[~/dnstunnelling]
$ sudo docker network ls
NETWORK ID          NAME                                DRIVER              SCOPE
05e84d4a772d        bridge                             bridge              local
26fd805b7875        dnstunnelling_lab-network          bridge              local
d1b4037883f3        host                               host                local
65015f8a725e        none                               null                local
```

Figure 4.2D: Docker network configuration

4.3 Parser Module

Scapy was used for PCAP reading and for sniffing in the live capture module, with a single API across both.

```
#DNS record type integers mapped to readable strings; scapy stores record types as numbers internally
RECORD_TYPES = {
    1: "A", #IPv4 address
    2: "NS", #Name Server
    5: "CNAME", #Canonical Name
    6: "SOA", #Start of Authority
    10: "NULL", #Null record, used for testing and debugging
    12: "PTR", #Pointer record, used for reverse DNS lookups
    15: "MX", #Mail Exchange record, used to specify mail servers for a domain
    16: "TXT", #Text record, used to store arbitrary text data associated with a domain
    28: "AAAA", #IPv6 address
    33: "SRV", #Service locator, used to define the location of servers for specified services
    255: "ANY", #Wildcard record, matches any record type
}
```

Figure 4.3A: Record Type Dictionary

```
def parse_pcap(filepath):
    # Read the pcap file and extract DNS query records.
    # Each record is returned as a dictionary with timestamp, query name, subdomain, parent_domain, and record_type.
    packets = rdpcap(filepath) # Read the pcap file and return a list of packets
    dns_queries = []
    for pkt in packets:
        if pkt.haslayer(DNS) and pkt[DNS].qr == 0: #Check if it's a DNS query
            queryname = pkt[DNSQR].qname.decode('latin-1').rstrip('.') #Iodine uses latin-1 encoding for domain names to preserve all byte values
            qtype = pkt[DNSQR].qtype #Get the DNS record type as an integer
            qtype_str = RECORD_TYPES.get(qtype, f"Unknown ({qtype})") #Convert the record type to a readable string, or label it as unknown if it's not in the mapping
            subdomain, parentdomain = extract_subdomain(queryname)

            #Build a dict
            record = {
                "timestamp": float(pkt.time),
                "query_name": queryname,
                "subdomain": subdomain,
                "parent_domain": parentdomain,
                "record_type": qtype_str,
            }
            dns_queries.append(record)
    return dns_queries
```

Figure 4.3B: PCAP parse function

The actual filtering logic shown in Figure 4.3B is as follows: each packet is inspected via *haslayer(DNS)*; only packets with a DNS layer and *qr == 0*, signifying a query (per RFC 1035 (Mockapetris, 1987)^[17]) are considered. DNS responses are excluded, since tunnelled payloads are carried in client-to-server queries, not responses (Wang et al., 2021)^[11].

Following this, the timestamp is extracted from *pkt.time*, full query name from the DNSQR layer's *qname* field, integer record type from *qtype*, which is then resolved to mnemonic via the dictionary shown in Figure 4.3A; the dictionary allows for extracted fields to be resolved to appropriate features (www.iana.org, n.d.)^[18].

It is worth noting that *qname* is decoded with *latin-1*; Iodine injects arbitrary 8-bit byte sequences into the subdomain field as part of the tunnelling method, and many of these sequences would raise a *UnicodeDecodeError*, hence the usage of *latin-1* (docs.python.org, n.d.)^[19].

The subdomain is split based on the dot, with a configurable *numlabels* parameter with a default of 2. It should be noted that domains using multiple labels in the Top-Level Domain (TLD) would be split incorrectly; for this project, this is not an issue as the traffic is controlled for testing, but future work should substitute in a library or method to split TLDs correctly in this instance.

The output schema is a list of dictionaries, each with five keys; using dictionaries rather than tuples helps with readability, as well as robustness, wherein adding a new field does not break any callers.

Additionally, unknown record types fall through to an “*Unknown (n)*” placeholder, essentially filtering out non-DNS records without raising errors.

```
(kali㉿kali)-[~/dnstunnelling]
$ python3 parser/parser.py data/iodine_tunnel.pcap | head -10
Subdomain: yrbdz5, Parent Domain: test.com, Record Type: NULL
Subdomain: vaaaaakaupia, Parent Domain: test.com, Record Type: NULL
Subdomain: laao5bci35bwh1jsk5xyrza2lszai4qi, Parent Domain: test.com, Record Type: NULL
Subdomain: yrbd0c, Parent Domain: test.com, Record Type: NULL
Subdomain: zd0daA-Aaahhh-Drink-mal-ein-Jägermeister-, Parent Domain: test.com, Record Type: NULL
Subdomain: zd0eaa-La-flûte-naïve-française-est-retiré-à-Crête, Parent Domain: test.com, Record Type: NULL
Subdomain: zd0faABcCdDeFFgGhHiIjJkKlLmMnNoOpPqQrRsStTuUvVwWxXyYzZ, Parent Domain: test.com, Record Type: NULL
Subdomain: zd0gaA01234567891011121314151617181920212223242526272829303132333435363738394041424344454647484950515253545556575859606162636465666768697071727374757677787980818283848586878889909192939495969798991001011021031041051061071081091101111121131141151161171181191201211221231241251261271281291301311321331341351361371381391401411421431441451461471481491501511521531541551561571581591601611621631641651661671681691701711721731741751761771781791801811821831841851861871881891901911921931941951961971981992002012022032042052062072082092102112122132142152162172182192202212222232242252262272282292302312322332342352362372382392402412422432442452462472482492502512522532542552562572582592602612622632642652662672682692702712722732742752762772782792802812822832842852862872882892902912922932942952962972982993003013023033043053063073083093103113123133143153163173183193203213223233243253263273283293303313323333343353363373383393403413423433443453463473483493503513523533543553563573583593603613623633643653663673683693703713723733743753763773783793803813823833843853863873883893903913923933943953963973983994004014024034044054064074084094104114124134144154164174184194204214224234244254264274284294304314324334344354364374384394404414424434444454464474484494504514524534544554564574584594604614624634644654664674684694704714724734744754764774784794804814824834844854864874884894904914924934944954964974984995005015025035045055065075085095105115125135145155165175185195205215225235245255265275285295305315325335345355365375385395405415425435445455465475485495505515525535545555565575585595605615625635645655665675685695705715725735745755765775785795805815825835845855865875885895905915925935945955965975985996006016026036046056066076086096106116126136146156166176186196206216226236246256266276286296306316326336346356366376386396406416426436446456466476486496506516526536546556566576586596606616626636646656666676686696706716726736746756766776786796806816826836846856866876886896906916926936946956966976986997007017027037047057067077087097107117127137147157167177187197207217227237247257267277287297307317327337347357367377387397407417427437447457467477487497507517527537547557567577587597607617627637647657667677687697707717727737747757767777787797807817827837847857867877887897907917927937947957967977987998008018028038048058068078088098108118128138148158168178188198208218228238248258268278288298308318328338348358368378388398408418428438448458468478488498508518528538548558568578588598608618628638648658668678688698708718728738748758768778788798808818828838848858868878888898908918928938948958968978988999009019029039049059069079089099109119129139149159169179189199209219229239249259269279289299309319329339349359369379389399409419429439449459469479489499509519529539549559569579589599609619629639649659669679689699709719729739749759769779789799809819829839849859869879889899909919929939949959969979989991000100110021003100410051006100710081009101010111012101310141015101610171018101910201021102210231024102510261027102810291030103110321033103410351036103710381039104010411042104310441045104610471048104910501051105210531054105510561057105810591060106110621063106410651066106710681069107010711072107310741075107610771078107910801081108210831084108510861087108810891090109110921093109410951096109710981099110011011102110311041105110611071108110911101111111211131114111511161117111811191120112111221123112411251126112711281129113011311132113311341135113611371138113911401141114211431144114511461147114811491150115111521153115411551156115711581159116011611162116311641165116611671168116911701171117211731174117511761177117811791180118111821183118411851186118711881189119011911192119311941195119611971198119912001201120212031204120512061207120812091210121112121213121412151216121712181219122012211222122312241225122612271228122912301231123212331234
```

Figure 4.3C: Example output of parser.py

4.4 Baseline Construction

The formulation of the statistical baseline is designed as a one-time computation that results in a static JSON artefact (Python Software Foundation, 2023)^[20]. The JSON is committed to the repository, so it does not need to be recreated, and the feature extractor has no overhead beyond a singular file read.

As stated previously, the data source is the Tranco top domains list (Le Pochat et al., 2019)^[10]. The sample size was selected as 10,000 domains, a somewhat arbitrary selection based on iterative testing discussed in §5, but sizes beyond this number prove to induce diminishing returns for increased computational overhead. This is configurable via the *num_domains* argument, however (Python Software Foundation, 2020)^[21].

```
# Fixed alphabet over which all distributions are defined
# Anything outside this alphabet is bucketed into "other"
# https://www.rfc-editor.org/rfc/rfc1035#section-2.3.1
ALPHABET = "abcdefghijklmnopqrstuvwxyz0123456789-"
OTHER = "other"
```

Figure 4.4A: Alphabet definitions

```
def count_chars_and_bigrams(text, char_counts, bigram_counts):
    # This function updates the provided char_counts and bigram_counts
    # Counters based on the characters and bigrams in the given text
    text = text.lower()

    for char in text:
        if char in ALPHABET: # Only count characters in the defined alphabet
            char_counts[char] += 1
        else:
            char_counts[OTHER] += 1

    # Counts bigrams by iterating through pairs of characters in the text
    for i in range(len(text) - 1):
        bigram = text[i:i + 2]
        if all(c in ALPHABET for c in bigram):
            bigram_counts[bigram] += 1
        else:
            bigram_counts[OTHER] += 1
```

Figure 4.4B: `count_chars_and_bigrams` function

The alphabet design (as seen in Figure 4.4A and utilised in 4.4B) is dictated by the ICANN domain rules (RFC 1035 §2.3.1 (Mockapetris, 1987)^[17]) and is a prerequisite for KL divergence in that both observed and baseline distributions must be defined over the same key space. In the absence of this constraint, KL divergence would be undefined whenever the observed distribution contained a character missing from the baseline.

The *other* bucket doubles as a fallback and holder for Iodine’s binary payloads, which produces a large mass at a single key; the effects of this will be further discussed in §7.

Every alphabet key is seeded with a zero count, essentially preventing missing keys in the final JSON, which would otherwise cause a *KeyError* when running feature extraction.

4.5 Feature Extraction Module

This module is structured as a set of individual calculation functions for each feature (as detailed in §3.5), integrated together by a master function (`compute_domain_features`).

Thresholds are defined as guards for whether calculations for a feature should be run or utilised based on the availability of data from the parser module.

```
SMOOTHING = 1e-10 # small constant added to probabilities to avoid zeros in KL divergence calculations
MIN_QUERIES_FOR_KL = 10 # minimum number of queries required to compute KL divergence
MIN_QUERIES_FOR_RATE = 10 # minimum number of queries required to compute rate features
MIN_DURATION_FOR_RATE = 1.0 # minimum duration (in seconds) required to compute rate features
MIN_CHARS_FOR_ENTROPY = 50 # minimum number of characters required to compute entropy
MIN_QUERIES_FOR_RATIO = 5 # minimum number of queries required to compute ratio features
MIN_QUERIES_FOR_FFT = 8 # minimum number of queries required to compute FFT features
```

Figure 4.5A: *Threshold guards*

These values were derived during iterative evaluation (see §5).

Distribution alignment for KL divergence is a necessary implementation because KL divergence is mathematically undefined when one distribution has zero probability for a symbol that the other gives positive probability (Darscheid, Guthke and Ehret, 2018)^[22]. Thus, the below function resolves this by taking the union of keys to ensure every symbol is represented in both arrays, then adding a small constant (as seen in Figure 4.5A) to every probability to remove zeroes.

```
def kl_divergence_aligned(observed, baseline):
    # Compute KL Divergence between two distributions that may have different keys by aligning them first
    all_keys = set(observed.keys()) | set(baseline.keys()) # Union of keys from both distributions

    obs_array = [observed.get(key, 0.0) + SMOOTHING for key in all_keys] #Align observed distribution to the full key set, filling in zeros (plus smoothing) for missing keys
    base_array = [baseline.get(key, 0.0) + SMOOTHING for key in all_keys] #same here

    #scipy.stats.entropy renormalises both arrays to sum to 1 internally
    return entropy(obs_array, base_array, base=2) #https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.entropy.html
```

Figure 4.5B: KL Divergence Alignment

Additionally, sample size guards are implemented on every feature whose statistical validity is dependent on sufficient data. If a check is failed, the feature is set to None so that the classifier does not treat that absence as a numerical value of zero, which would influence classification and skew results towards benign and false negative results. To complement this design choice, a list of flags is maintained for each skipped feature to also produce a readable reason, keeping in line with the defined problem definition in §1.2 for the purposes of traceability.

The full `compute_domain_features` function is documented and can be viewed on the accompanying repository for this report for a more in-depth breakdown of mathematical functions; explaining each one here would be redundant.

Furthermore, the baseline distribution JSON is loaded once with an explicit path relative to the module file, not the call location, which, provided that the main project directory is not manipulated, makes this module functional independent of location.

```
(kali@kali) - [~/dnstunnelling]
$ python3 features/featureextraction.py data/iodine_tunnel.pcap
Parsed 104 DNS records

=== test.com ===
shannon_entropy           : 6.0356
char_kl_divergence        : 11.5734
bigram_kl_divergence      : 22.1319
mean_subdomain_length    : 40.0192
stddev_subdomain_length  : 77.7145
total_volume              : 104
query_rate                : 0.3277
unique_subdomains         : 104
unique_subdomain_ratio    : 1.0000
uncommon_record_proportion : 1.0000
peak_to_median_ratio      : 4.6871
flags                     : []
```

Figure 4.5C: Feature Extraction Output

4.6 FFT Module

NumPy provides an implementation of the FFT algorithm, which requires evenly spaced input samples, and thus, the pipeline performs a resampling step before any frequency analysis. This is handled by `numpy.histogram`, which buckets timestamps into bins to produce an evenly spaced array (Harris et al., 2020; numpy.org, n.d.)^{[11][23]}.

```
num_bins = max(1, int(np.ceil(duration / BIN_SIZE))) # Step 1
counts, _ = np.histogram(timestamps, bins=num_bins, range=(timestamps[0], timestamps[0] + num_bins * BIN_SIZE))
```

Figure 4.6A: Histogram for converting timings into a binned spectrum

Given that the input count is an array, RFFT (real-valued FFT) is used over FFT in NumPy; RFFT returns only the half of the spectrum that is not redundant; the nature of the histogram buckets means that the second half is symmetrical, and so RFFT can be used to reduce computational overhead and avoid meaningless duplicate frequencies in the output (numpy.org, n.d.; Sorensen et al., 1987)^{[24][25]}.

```

dominant_bin = ac.argmax() + 1 #+1 because index 0 was dropped
freqs = np.fft.rfftfreq(num_bins, d=BIN_SIZE)
dominant_freq = freqs[dominant_bin] #Hz
period = 1.0 / dominant_freq if dominant_freq > 0 else float('inf') # Step 5.

return { #Return all the relevant information about the FFT analysis for this
    "num_queries": len(timestamps),
    "duration_s": duration,
    "peak": peak,
    "median_ac": median,
    "ratio": ratio,
    "dominant_period_s": period,
    "is_beacon": ratio >= BEACON_THRESHOLD,
}

```

Figure 4.6B: RFFT Application

The frequency bin of zero on an FFT encodes the mean of the input signal, i.e., a measure of average query rate, not periodicity. To include it would dominate the peak and produce false positives for any domain with a high volume regardless of timing structure (Kim et al., 2016)^[6].

The absolute peak magnitude scales with input volume, essentially, a domain with high volume has a large peak even if it does not beacon; dividing by the median of the spectrum normalises against the spectrum's baseline. Median is opted for over the mean since a single sharp peak would skew the mean.

```

# Thresholds similar to the feature extractor
BIN_SIZE = 1.0 # seconds per bin, the width is used to resample the query timings into an evenly=space time grid
BEACON_THRESHOLD = 4.0 # peak:median ratio above which we flag beaconing
MIN_QUERIES = 8 # skip domains with too few queries to analyse

```

Figure 4.6C: Defined Constants for Guards and Thresholds

The bin size is defined as 1 second; the Nyquist limit means this resolves periods down to roughly 2 seconds, which comfortably covers the keepalive intervals observed in DNS tunnelling tools. A smaller bin width would resolve faster beacons but produce a noisier histogram, whereas a larger bin would let faster periodicities smooth out entirely.

The sample size guard is set to 8; below this, the signal is too sparse to produce a meaningful spectrum. This value is also set in the feature extractor (§3.5), so that the two modules agree on what is sufficient data. Additionally, unusable inputs, which may be caused by captures with no durations, or captures that produce only one bin, a None value is returned, as previously discussed.

4.7 Classifier Module

Per the proposed classification methodology in §3.8, weights, normalisation and classification thresholds exist in a configuration dictionary; configuring weights and thresholds becomes much easier if centralised in this manner.

```

DEFAULT_CONFIG = {
    "features": {
        #Has already been normalised, proportional 0-1 scale
        "uncommon_record_proportion": {"low": 0.0, "high": 1.0, "weight": 3.0},
        #Iodine produces much higher values for Char KL,
        # and is somewhat of an outlier. Despite that, it is well above the threshold by a significant margin,
        # and will produce an equivalently high suspicion score. Hence, the high threshold is set to a lower
        # value than what a tool like Iodine produces.
        "char_kl_divergence": {"low": 1.5, "high": 3.0, "weight": 2.5},
        #Unique:total distribution of subdomains. Higher means more unique subdomains, which is more suspicious.
        "unique_subdomain_ratio": {"low": 0.7, "high": 0.95, "weight": 2.0},
        #Shannon entropy of the subdomains. Higher means more random-looking, which is more suspicious.
        "shannon_entropy": {"low": 3.5, "high": 6.0, "weight": 1.5},
        #Average length of subdomains. Higher means more random-looking, which is more suspicious.
        "mean_subdomain_length": {"low": 15.0, "high": 35.0, "weight": 1.5},
        #This is less discriminative than other features, because it could also just be behaviour of a regular user clicking on lots of webpages.
        "query_rate": {"low": 1.0, "high": 5.0, "weight": 1.0},
        #Has much more overlap between benign and tunnel domains, so it has a low weight and a higher threshold.
        "bigram_kl_divergence": {"low": 6.0, "high": 8.0, "weight": 0.5},
        #This is the beacon catching, it will only capture certain tools though; not all beacon, but when they do, it is a strong indicator.
        "peak_to_median_ratio": {"low": 2.5, "high": 5.0, "weight": 1.5},
    },
    "thresholds": {
        #Score cutoffs as defined through iterative testing.
        "tunnel": 0.7,
        "suspicious": 0.4,
    },
    #Min number of features required to output anything but "insufficient_data"; stops reliance on a single feature.
    "min_features_for_class": 3,
}

```

Figure 4.7A: Default Configuration Dictionary

The normalise function performs linear interpolation, ranging from 0-1 to prevent any single feature skewing results with an extreme value; i.e., a feature massively above the defined “high” threshold will contribute the same as a feature that just meets that threshold so as to not affect the total weighted sum.

```

def normalise(value, low, high):
    #Map a val from the inconsistent [low, high] range to [0,1]
    # This is private and not part of the public API - PEP8: https://peps.python.org/pep-0008/#naming-conventions

    #Guard because high==low would divide by zero
    if high == low:
        return 0.0
    return max(0.0, min(1.0, (value - low) / (high - low)))

```

Figure 4.7B: Normalise function

Additionally, a feature that returns None after feature extraction (§4.5) will be skipped entirely, where its contribution to the weighted sum (the numerator for the final score), and the total weight (the denominator for the final score) is removed. Furthermore, there is a specific guard `min_features_for_class` which prevents classification if there is not enough data to make a reasonable verdict. If fewer than this many features are contributed, then the class comes out as a separate “insufficient_data” result.


```

for name, cfg in feature_cfgs.items():
    #skip all the missing ones; the score has been normalised to be between 0 and 1
    value = features.get(name)
    if value is None:
        continue

    suspicion = _normalise(value, cfg["low"], cfg["high"])
    weight = cfg["weight"]

    weighted_sum += weight * suspicion
    total_weight += weight
    contributions[name] = {"raw": value, "suspicion": suspicion, "weight": weight}

features_used = len(contributions)

#A guard for insufficient data; without enough data, any output would be statistical
if features_used < min_features:
    return {
        "score": None,
        "classification": "insufficient_data",
        "features_used": features_used,
        "contributing_features": contributions,
    }

score = weighted_sum / total_weight

```

Figure 4.7C: Weight Calculations and Insufficient Data Guard

As proposed in §3.8, the classifier will, excepting the above case for insufficient data, produce a classification of either tunnel, suspicious or benign. Each classification output is complemented by a dictionary of the features that contributed to that output, and what their scores were, per the necessity for explainability discussed in §1.2.

```

#The classification!
if score >= thresholds["tunnel"]:
    classification = "tunnel"
elif score >= thresholds["suspicious"]:
    classification = "suspicious"
else:
    classification = "benign"

```

Figure 4.7D: Classification

```

def classify_all(all_features, config=None):
    #Run through all the domains and classify them!
    return {
        domain: classify_domain(features, config) for domain, features in all_features.items()
    }

```

Figure 4.7E: Classify All

It should be noted that in the main function that simply iterates through all the domains and features and calls the above classification logic defaults the *config* parameter to None, and

then to `DEFAULT_CONFIGS` (Figure 4.7A) inside the function; directly initialising `configs` to `DEFAULT_CONFIGS` would cause that single, same instance of that dictionary to be used across all calls (Python-guide.org, 2024)^[26].

4.8 Live Capture

This is the live counterpart to the above static classifier (§4.7). The only difference is the input source; the overarching pipeline is the same. The necessary modules are imported internally, so any changes to those individual components will take effect in the live classifier automatically.

```
sniffer_kwargs = {
    "filter": "udp port 53", #BPF filter to capture only DNS queries pcap-filter(7): https://www.tcpdump.org/manpages/pcap-filter.7.html
    "prn": _packet_callback, #accesses the callback function
    "store": False, #tells scapy not to retain any captured packets, otherwise, AsyncSniffer would accumulate every packet in memory
}
if args.interface:
    sniffer_kwargs["iface"] = args.interface
sniffer = AsyncSniffer(**sniffer_kwargs)
sniffer.start()
```

Figure 4.8A: AsyncSniffer

Scapy's AsyncSniffer provides both synchronous and asynchronous interfaces for packet sniffing; the asynchronous variant is used so that the sniffer can run a background thread, which is occasionally recorded, a fixed time-window is copied, and handed off to a main thread that performs the above analysis pipeline (§4.3-4.7; excluding the baseline calculator, of course) (Biondi, 2026)^[9].

The sniffer is configured to run on UDP port 53 with Berkeley Packet Filter (BPF) syntax. BPF filters are compiled and applied inside the kernel, so packets that aren't DNS never cross the boundary into python (McCanne and Jacobson, 1992)^[27].

```
def _packet_callback(pkt) -> None:
    #sniffer callback; what to actually do with each packet the sniffer finds.
    record = _packet_to_record(pkt)

    #skip if None
    if record is None:
        return

    #the lock is held only until the record is appended
    with _lock:
        _records.append(record)
```

Figure 4.8B: Callback Function

By default, Scapy retains every packet in memory; this would be both inefficient and insecure, so Scapy is instructed to discard each packet after the callback (the function that is invoked each time Scapy sniffs a packet) returns, retaining only the records of analysis (Biondi, 2026)^[9].

```
#these make a shared packet buffer
# records is the list of parsed records between the sniffer thread and the thread doing all the classification
# lock is used to coordinate access to the shared records list; the sniffer thread acquires the lock to append new records
# the main thread acquires it to read recent records and to discard old ones
_lock = threading.Lock()
_records: list[dict] = []
```

Figure 4.8C: Shared record and lock definition

Two threads share a single records buffer protected by a lock. The sniffer thread appends parsed information to this record buffer using the parser module (§4.3). The main thread sleeps while this happens, then removes the last records it was working on, and then analyses the most recently appended records. The lock is held only for the append operation on the sniffer thread, and only for the read and remove operation on the main analysis thread. Given that the copy operation by the main analysis thread is taken under the lock and analysed outside of it, the lock time is constrained regardless of how long analysis takes. Additionally, the baseline dictionary is loaded once before threads start and is then read-only, so can be freely shared between threads (Python Software Foundation, 2025)^[28].

```
def _run_analysis(
    window_secs: int, #how big is the window
    baseline: dict, #the reference baseline
    config: dict, #the configuration for the classifier
    threshold_verdict: str, #the minimum verdict level to alert on
    verbose: bool,
) -> None:

    now = time.time()
    cutoff = now - window_secs

    #prune the buffer to just the recent records, and take a copy of them for analysis;
    with _lock:
        _records[:] = [r for r in _records if r["timestamp"] >= cutoff]
        snapshot = list(_records)

    ts = _timestamp()

    #if there is no traffic, just put a message and skip the analysis.
    if not snapshot:
        print(f"[{ts}] No DNS queries seen in the last {window_secs}s, waiting...")
        return

    #compute features and classify them!
    features = compute_features(snapshot, baseline)
    results = classify_all(features, config)
```

Figure 4.8D: Main Analysis Thread

The mechanics of the actual remove and copy operation is as follows: records older than current time minus time window duration are discarded; essentially creating a rolling time window that lags slightly behind live capture, analysing them, and then moving on. This means that a tunnel is detected within the length of the time interval that the main analysis thread sleeps once enough records have been accumulated to satisfy the guards on analysis discussed in §4.5 onward.

```
def _packet_to_record(pkt) -> dict | None:
    #convert a single packet into a record dictionary
    # if the packet isn't a DNS query, or if it doesn't have a parsable query name, return None to skip it
    if not (pkt.haslayer(DNS) and pkt[DNS].qr == 0):
        return None

    #the qname decode is a try/except because in Scapy, malformed packets do occasionally get through and would otherwise cause a crash
    try:
        queryname = pkt[DNSQR].qname.decode("latin-1").rstrip(".")
    except Exception:
        return None

    #qtype is an integer DNS record, DNSQR turns it into the conventional mnemonic (A, TXT, NULL, etc.)
    qtype = pkt[DNSQR].qtype
    subdomain, parentdomain = extract_subdomain(queryname)

    return {
        "timestamp": float(pkt.time),
        "query_name": queryname,
        "subdomain": subdomain,
        "parent_domain": parentdomain,
        "record_type": RECORD_TYPES.get(qtype, f"Unknown ({qtype})"),
    }
```

Figure 4.8E: Function that reads records from a single packet

It should be noted that Scapy sometimes allows malformed packets through; the BPF filter will allow DNS packets through but does not check if they are malformed. Thus, the initial packet reading (Figure 4.8E) is wrapped in a *try/except* to handle these gracefully (Biondi, 2026)^[9].

4.9 Evaluation

An evaluation script was written essentially as a wrapper for the static classification tool (§4.7), where PCAP files are sequentially classified, but with the addition of a “true class” dictionary that statically maps known class to the PCAP test data being evaluated.

```

TRUE_CLASS = { #Works as true labels for the evaluation
  "data/iodine_tunnel.pcap": "tunnel",
  "data/dns2tcp_tunnel.pcap": "tunnel",
  "data/benign.pcap": "benign",

  #benign
  "data/benign/browsing.pcap": "benign",
  "data/benign/streaming.pcap": "benign",
  "data/benign/development.pcap": "benign",
  "data/benign/mixed.pcap": "benign",

  #iodine
  "data/iodine/null_idle.pcap": "tunnel",
  "data/iodine/txt_idle.pcap": "tunnel",
  "data/iodine/null_active.pcap": "tunnel",
  "data/iodine/alt_domain.pcap": "tunnel",

  #dns2tcp
  "data/dns2tcp/default_light.pcap": "tunnel",
  "data/dns2tcp/default_heavy.pcap": "tunnel",
  "data/dns2tcp/alt_domain.pcap": "tunnel",
  "data/dns2tcp/sustained.pcap": "tunnel",

  #DNSScat2
  "data/dnscat2/idle.pcap": "tunnel",
  "data/dnscat2/interactive.pcap": "tunnel",
  "data/dnscat2/exfil.pcap": "tunnel",
  "data/dnscat2/alt_domain.pcap": "tunnel",
}

```

Figure 4.9.1A: True Class Dictionary

As each PCAP is run through the static classifier, a count of True Positive, False Positive, True Negative and False Negative is maintained using a simple else-if block:

```

tp = 0
fp = 0
tn = 0
fn = 0
domain_outcomes = []

for domain, result in results.items():
    predicted = class_to_label(result["classification"])

    if expected_label == "tunnel" and predicted == "tunnel":
        outcome = "TP"
        tp += 1
    elif expected_label == "tunnel" and predicted == "benign":
        outcome = "FN"
        fn += 1
    elif expected_label == "benign" and predicted == "tunnel":
        outcome = "FP"
        fp += 1
    else:
        outcome = "TN"
        tn += 1

```

Figure 4.9.2B: Else-If Block

Where those counts are fed into calculations for precision, recall and F1:

```

tp = sum(r["tp"] for r in per_pcap)
fp = sum(r["fp"] for r in per_pcap)
tn = sum(r["tn"] for r in per_pcap)
fn = sum(r["fn"] for r in per_pcap)

print("\nConfusion matrix")
print(f"          {'Pred. tunnel':>14} {'Pred. benign':>14}")
print(f" Actually tunnel  {tp:>14}  {fn:>14}")
print(f" Actually benign  {fp:>14}  {tn:>14}")

precision = safe_div(tp, tp + fp)
recall    = safe_div(tp, tp + fn)
f1 = (
    safe_div(2 * precision * recall, precision + recall)
    if precision is not None and recall is not None
    else None
)

print("\nMetrics")
print(f" Precision: {fmt(precision)}")
print(f" Recall:    {fmt(recall)}")
print(f" F1 score:   {fmt(f1)}")

```

Figure 4.9.3C: Precision, Recall and F1 calculations

Each of these calculations is guarded by a safe division function, where a try/catch statement is abstracted into a separate function “*safe_div*” so as to both avoid division by zero, and avoid cluttering the code by typing an individual try/catch block for each calculation.

```
def safe_div(numerator: float, denominator: float) -> float | None:
    #This is to avoid ever dividing by zero. Made it into a function because otherwise we'd have to have a lot of try/catch blocks lower down
    if denominator == 0:
        return None
    else:
        return numerator / denominator
```

Figure 4.9.3D: Safe Division Function

```
per_pcap = [] #Keeps track of the results for each PCAP separately
for pcap_path, expected_label in TRUE_CLASS.items():
    full_path = os.path.join(SCRIPT_DIR, pcap_path)
    if not os.path.exists(full_path):
        print(f"ERROR: missing pcap {full_path}", file=sys.stderr)
        sys.exit(1)
    per_pcap.append(evaluate_pcap(full_path, expected_label)) #Adds the results for one PCAP to the list, so we can keep track of which PCAP got which result
```

Figure 4.9.3E: Main Evaluation Loop

Figure 4.9.3E shows the main evaluation loop, where a list of dictionaries produced by the evaluation function is maintained, so that a comprehensive breakdown of how each file was classified can be viewed.

5 Testing and Validation

5.1 Testing Methodology

The iterative testing process will occur in three broad phases; firstly, validation tests on each module individually, since the proposed pipeline in §3.1 is linear, and each stage’s output is the next stage’s input; second, testing the full pipeline across three control PCAP files to verify that each module works sequentially; lastly, iterative tuning of thresholds, such as guards, normalisation ranges, classification thresholds, etc.

As discussed in §3.9, precision, recall and F1 will be the primary metrics used to assess any parameter tuning (Liu et al., 2017)^[13].

5.2 Test Data

The data for the proposed methodology in §5.1 consists of three PCAP files; one benign, and two generated by the proposed tunnelling tools in §3.3. All three were generated and captured under the conditions and tools discussed in §3.3, with the environment implemented in §4.2.

All three PCAP files are committed to the repository under *data/*, per §4.1. To generate traffic with Iodine (the domain name can be anything):

1. docker exec -it dns-attacker bash (Terminal 1)
2. iodined -f 10.0.0.1 test.com (Terminal 1)
 - a. enter any password
3. docker exec -it dns-victim bash (Terminal 2)
4. iodine -f -r 172.20.0.10 test.com (Terminal 2)
 - a. enter the same password as above

To generate traffic with dns2tcp (Requires three terminals):

1. docker exec -it dns-attacker bash (Terminal 1)
2. dns2tcpd -F -d 1 -f /etc/dns2tcpd.conf (Terminal 1)

- Testing of the live capture module will occur with a live usage of the proposed tunnelling tools, as well as some benign browsing behaviour.

Below is a series of screenshots of terminal output, showing successful execution of each implemented module ([§4.3-4.8](#)) across all three PCAP files and live techniques discussed in [§5.2](#).

```
(kali@kali)-[~/dnstunnelling]
$ python3 parser/parser.py data/benign.pcap | head -10
Subdomain: firefox.settings.services, Parent Domain: mozilla.com, Record Type: A
Subdomain: ads-img, Parent Domain: mozilla.org, Record Type: A
Subdomain: github, Parent Domain: githubassets.com, Record Type: A
Subdomain: github, Parent Domain: githubassets.com, Record Type: AAAA
Subdomain: avatars, Parent Domain: githubusercontent.com, Record Type: A
Subdomain: avatars, Parent Domain: githubusercontent.com, Record Type: AAAA
Subdomain: , Parent Domain: github.com, Record Type: A
Subdomain: telemetry.individual, Parent Domain: githubcopilot.com, Record Type: A
Subdomain: telemetry.individual, Parent Domain: githubcopilot.com, Record Type: AAAA
Subdomain: github, Parent Domain: githubassets.com, Record Type: A
```

```
(kali㉿kali)-[~/dnstunnelling]
$ python3 parser/parser.py data/iodine_tunnel.pcap | head -10
Subdomain: yrbdz5, Parent Domain: test.com, Record Type: NULL
Subdomain: vaaaakaupia, Parent Domain: test.com, Record Type: NULL
Subdomain: laao5bcj35bwihjsk5xyrza2lsczi4qi, Parent Domain: test.com, Record Type: NULL
Subdomain: yrbd0c, Parent Domain: test.com, Record Type: NULL
Subdomain: zd0daA-Aaaahhh-Drink-mal-ein-Jägermeister-, Parent Domain: test.com, Record Type: NULL
Subdomain: zd0eaA-La-flûte-naïve-française-est-retiré-à-Crête, Parent Domain: test.com, Record Type: NULL
Subdomain: zd0faAbBcCdDeEfFgGhHiIjJkKlLmMnNoOpPqQrRsStTuUvVwWxXyYzZ, Parent Domain: test.com, Record Type: NULL
Subdomain: zd0gaAo12345678910111213AAAABBBCCDEEEIII, Parent Domain: test.com, Record Type: NULL
Subdomain: zd0haADN000000×UUUUUYPBaaaaaaæcééeéiiiiidñóóóó×uuúüý, Parent Domain: test.com, Record Type: NULL
Subdomain: sahd0i, Parent Domain: test.com, Record Type: NULL
```

```
(kali㉿kali) ~/dnstunnelling
$ python3 parser/parser.py data/dns2tcp_tunnel.pcap | head -10
Subdomain: AAAAABY5AA.=auth, Parent Domain: tunnel.test, Record Type: TXT
Subdomain: fCygAABADUwRERENzdFMjYwMzg3MjA0UUyQURBMzZDMjVFMzgm0VENDVEMKE.=auth, Parent Domain: tunnel.test, Record Type: TXT
Subdomain: fCw8+26oAHNzaA.=connect, Parent Domain: tunnel.test, Record Type: TXT
Subdomain: AAAAANwUA.=auth, Parent Domain: tunnel.test, Record Type: TXT
Subdomain: fyyFgAABADVFMzJEMDJENkM1MKZDNTEwNTM1QzVFMzlerDE2QURCOEIzqzJDNZa.=auth, Parent Domain: tunnel.test, Record Type: TXT
Subdomain: fyz7mjRNAHNzaA.=connect, Parent Domain: tunnel.test, Record Type: TXT
Subdomain: AAAAACdAAA.=auth, Parent Domain: tunnel.test, Record Type: TXT
Subdomain: iAaFgAABAE5UUzMDREM0VGQzJBNDcxRUIyNUNEMzZDMUyVjc1NEFDNJGNEI.=auth, Parent Domain: tunnel.test, Record Type: TXT
Subdomain: iAYMKIp2AHNzaA.=connect, Parent Domain: tunnel.test, Record Type: TXT
Subdomain: AAAAAHoFAA.=auth, Parent Domain: tunnel.test, Record Type: TXT
```

The below figure shows the extracted features for only one domain and is representative; the full output is very long.

```

(kali@kali)-[~/dnstunnelling]
$ python3 features/featureextraction.py data/benign.pcap

=== mozilla.com ===
shannon_entropy          : None
char_kl_divergence       : None
bigram_kl_divergence     : None
mean_subdomain_length   : 25.0000
stddev_subdomain_length : 0
total_volume             : 1
query_rate               : None
unique_subdomains        : 1
unique_subdomain_ratio   : None
uncommon_record_proportion : 0.0000
peak_to_median_ratio     : None
flags                   : ['entropy_skipped_low_chars', 'kl_skipped_low_volume', 'unique_ratio_skipped_low_volume', 'fft_skipped_low_volume']

```

Figure 5.3.2A: Feature Extraction on Benign PCAP

```

(kali@kali)-[~/dnstunnelling]
$ python3 features/featureextraction.py data/iodine_tunnel.pcap
Parsed 104 DNS records

=== test.com ===
shannon_entropy          : 6.0356
char_kl_divergence       : 11.5734
bigram_kl_divergence     : 22.1319
mean_subdomain_length   : 40.0192
stddev_subdomain_length : 77.7145
total_volume             : 104
query_rate               : 0.3277
unique_subdomains        : 104
unique_subdomain_ratio   : 1.0000
uncommon_record_proportion : 1.0000
peak_to_median_ratio     : 4.6871
flags                   : []

```

Figure 5.3.2B: Feature Extraction on Iodine PCAP

```

(kali@kali)-[~/dnstunnelling]
$ python3 features/featureextraction.py data/dns2tcp_tunnel.pcap
Parsed 1269 DNS records

=== tunnel.test ===
shannon_entropy          : 5.3537
char_kl_divergence       : 2.2248
bigram_kl_divergence     : 6.6221
mean_subdomain_length   : 36.0000
stddev_subdomain_length : 23.5089
total_volume             : 1269
query_rate               : 2.9916
unique_subdomains        : 1268
unique_subdomain_ratio   : 0.9992
uncommon_record_proportion : 1.0000
peak_to_median_ratio     : 1.4168
flags                   : []

```

Figure 5.3.2C: Feature Extraction on dns2tcp PCAP

5.3.3 FFT Tests

```
(kali㉿kali)-[~/dnstunnelling]
$ python3 fft/fft.py data/benign.pcap

=====
FFT beaoning analysis: data/benign.pcap
=====
Domain                Queries  Period(s)   Ratio  Flag
-----
gstatic.com           15      5.3        2.6
google.com            31     10.3        1.6
co.uk                  8       3.0        1.0

Beaoning domains (0): none
```

Figure 5.3.3A: FFT Test on Benign PCAP

```
(kali㉿kali)-[~/dnstunnelling]
$ python3 fft/fft.py data/iodine_tunnel.pcap

=====
FFT beaoning analysis: data/iodine_tunnel.pcap
=====
Domain                Queries  Period(s)   Ratio  Flag
-----
test.com              104      2.0        4.7 *** BEACON ***

Beaoning domains (1): test.com
```

Figure 5.3.3B: FFT Test on Iodine PCAP

```
(kali㉿kali)-[~/dnstunnelling]
$ python3 fft/fft.py data/dns2tcp_tunnel.pcap

=====
FFT beaoning analysis: data/dns2tcp_tunnel.pcap
=====
Domain                Queries  Period(s)   Ratio  Flag
-----
tunnel.test           1269      4.8        1.4

Beaoning domains (0): none
```

Figure 5.3.3C: FFT Test on dns2tcp PCAP

5.3.4 Classifier Tests

Figure 5.3.4A (below) is a representative sample; the full output is very long.

```
(kali㉿kali)-[~/dnstunnelling]
$ python3 classifier/classifier.py data/benign.pcap

===== mozilla.com =====
score=N/A classification=insufficient data features_used=2
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
mean_subdomain_length      : raw=25.0000 suspicion=0.5000 weight=1.5

===== mozilla.org =====
score=N/A classification=insufficient data features_used=2
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
mean_subdomain_length      : raw=7.0000 suspicion=0.0000 weight=1.5

===== githubassets.com =====
score=0.0000 classification=benign features_used=3
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
unique_subdomain_ratio      : raw=0.2000 suspicion=0.0000 weight=2.0
mean_subdomain_length      : raw=6.0000 suspicion=0.0000 weight=1.5

===== githubusercontent.com =====
score=N/A classification=insufficient data features used=2
```

Figure 5.3.4A: Classifier Test on Benign PCAP

```
(kali㉿kali)-[~/dnstunnelling]
$ python3 classifier/classifier.py data/iodine_tunnel.pcap

===== test.com =====
score=0.9120 classification=tunnel features_used=8
uncommon_record_proportion : raw=1.0000 suspicion=1.0000 weight=3.0
char_kl_divergence         : raw=11.5734 suspicion=1.0000 weight=2.5
unique_subdomain_ratio     : raw=1.0000 suspicion=1.0000 weight=2.0
shannon_entropy            : raw=6.0356 suspicion=1.0000 weight=1.5
mean_subdomain_length      : raw=40.0192 suspicion=1.0000 weight=1.5
query_rate                 : raw=0.3277 suspicion=0.0000 weight=1.0
bigram_kl_divergence       : raw=22.1319 suspicion=1.0000 weight=0.5
peak_to_median_ratio       : raw=4.6871 suspicion=0.8749 weight=1.5
```

Figure 5.3.4B: Classifier Test on Iodine

```
(kali㉿kali)-[~/dnstunnelling]
$ python3 classifier/classifier.py data/dns2tcp_tunnel.pcap

===== tunnel.test =====
score=0.7018 classification=tunnel features_used=8
uncommon_record_proportion : raw=1.0000 suspicion=1.0000 weight=3.0
char_kl_divergence         : raw=2.2248 suspicion=0.4832 weight=2.5
unique_subdomain_ratio     : raw=0.9992 suspicion=1.0000 weight=2.0
shannon_entropy            : raw=5.3537 suspicion=0.7415 weight=1.5
mean_subdomain_length      : raw=36.0000 suspicion=1.0000 weight=1.5
query_rate                 : raw=2.9916 suspicion=0.4979 weight=1.0
bigram_kl_divergence       : raw=6.6221 suspicion=0.3110 weight=0.5
peak_to_median_ratio       : raw=1.4168 suspicion=0.0000 weight=1.5
```

Figure 5.3.4C: Classifier Test on dns2tcp

5.3.5 Live Testing

Figure 5.3.5A was produced as a result of benign browsing activity on the host VM whilst the live detector was running.

```
(kali@kali)~/dnstunnelling
$ sudo python3 detector/live.py --interface eth0 --window 60 --interval 10 --threshold suspicious --verbose
[12:06:58] DNS tunnel detector: live capture mode
[12:06:58]   interface : eth0
[12:06:58]   window   : 60s
[12:06:58]   interval  : 10s
[12:06:58]   threshold : suspicious
[12:06:58]   Press Ctrl+C to stop.

[12:07:08] Analysed 4 domain(s) in window, nothing above threshold
[12:07:18] Analysed 10 domain(s) in window, nothing above threshold
[12:07:28] Analysed 19 domain(s) in window, nothing above threshold
[12:07:38] Analysed 22 domain(s) in window, nothing above threshold
[12:07:48] Analysed 22 domain(s) in window, nothing above threshold
^C
[12:07:53] Stopping.
[12:07:53] Capture stopped. Goodbye.
```

Figure 5.3.5A: Benign Live Detection

```
(kali@kali)~/dnstunnelling
$ sudo python3 detector/live.py --interface br-26fd805b7875 --window 60 --interval 10 --threshold suspicious --verbose
[12:09:56] =====
[12:09:56] 1 ALERT(S) - 1 domain(s) analysed
[12:09:56] =====
[12:09:56] [TUNNEL ] score=0.850 features=8 test.com
[12:09:56] uncommon_record_proportion raw= 1.0000 suspicion=1.000 weight=3.0
[12:09:56] char_kl_divergence raw= 7.1695 suspicion=1.000 weight=2.5
[12:09:56] unique_subdomain_ratio raw= 1.0000 suspicion=1.000 weight=2.0
[12:09:56] shannon_entropy raw= 5.1266 suspicion=0.651 weight=1.5
[12:09:56] mean_subdomain_length raw= 95.1250 suspicion=1.000 weight=1.5
[12:09:56] query_rate raw= 7.9432 suspicion=1.000 weight=1.0
[12:09:56] bigram_kl_divergence raw= 14.1857 suspicion=1.000 weight=0.5
[12:09:56] peak_to_median_ratio raw= 1.0000 suspicion=0.000 weight=1.5

[12:10:06] =====
[12:10:06] 1 ALERT(S) - 1 domain(s) analysed
[12:10:06] =====
[12:10:06] [TUNNEL ] score=0.827 features=8 test.com
[12:10:06] uncommon_record_proportion raw= 1.0000 suspicion=1.000 weight=3.0
[12:10:06] char_kl_divergence raw= 8.6023 suspicion=1.000 weight=2.5
[12:10:06] unique_subdomain_ratio raw= 1.0000 suspicion=1.000 weight=2.0
[12:10:06] shannon_entropy raw= 5.5021 suspicion=0.801 weight=1.5
[12:10:06] mean_subdomain_length raw=131.3333 suspicion=1.000 weight=1.5
[12:10:06] query_rate raw= 2.8361 suspicion=0.459 weight=1.0
[12:10:06] bigram_kl_divergence raw= 16.1758 suspicion=1.000 weight=0.5
[12:10:06] peak_to_median_ratio raw= 1.3269 suspicion=0.000 weight=1.5

^C
[12:10:13] Stopping.
[12:10:13] Capture stopped. Goodbye.
```

Figure 5.3.5B: Live Detection on Iodine Tunnel

```

(kali@kali)-[~/dnstunnelling]
$ sudo python3 detector/live.py --interface br-26fd805b7875 --window 60 --interval 10 --threshold suspicious --verbose
[12:13:47] Press Ctrl+C to stop.

[12:13:57] =====
[12:13:57] 1 ALERT(S) - 1 domain(s) analysed
[12:13:57] =====
[12:13:57] [TUNNEL ] score=0.760 features=8 tunnel.test
[12:13:57] uncommon_record_proportion raw= 1.0000 suspicion=1.000 weight=3.0
[12:13:57] char_kl_divergence raw= 2.4483 suspicion=0.632 weight=2.5
[12:13:57] unique_subdomain_ratio raw= 1.0000 suspicion=1.000 weight=2.0
[12:13:57] shannon_entropy raw= 5.3148 suspicion=0.726 weight=1.5
[12:13:57] mean_subdomain_length raw= 36.0000 suspicion=1.000 weight=1.5
[12:13:57] query_rate raw= 3.4844 suspicion=0.621 weight=1.0
[12:13:57] bigram_kl_divergence raw= 7.8874 suspicion=0.944 weight=0.5
[12:13:57] peak_to_median_ratio raw= 1.0000 suspicion=0.000 weight=1.5

[12:14:07] =====
[12:14:07] 1 ALERT(S) - 1 domain(s) analysed
[12:14:07] =====
[12:14:07] [TUNNEL ] score=0.742 features=8 tunnel.test
[12:14:07] uncommon_record_proportion raw= 1.0000 suspicion=1.000 weight=3.0
[12:14:07] char_kl_divergence raw= 2.4090 suspicion=0.606 weight=2.5
[12:14:07] unique_subdomain_ratio raw= 1.0000 suspicion=1.000 weight=2.0
[12:14:07] shannon_entropy raw= 5.3399 suspicion=0.736 weight=1.5
[12:14:07] mean_subdomain_length raw= 36.0000 suspicion=1.000 weight=1.5
[12:14:07] query_rate raw= 3.1753 suspicion=0.544 weight=1.0
[12:14:07] bigram_kl_divergence raw= 7.4385 suspicion=0.719 weight=0.5
[12:14:07] peak_to_median_ratio raw= 1.3333 suspicion=0.000 weight=1.5

```

Figure 5.3.5C: Live Detection on dns2tcp

5.4 Threshold Tuning

The initial thresholds were somewhat arbitrary selections based on existing literature and were subsequently refined based on iterative testing.

5.4.1 Feature Extraction Tuning

Initially, the minimum sample size guards were set to be too small; without sufficiently high values, benign domains produce massively inflated scores that cause false positives.

```

SMOOTHING = 1e-10 # small c
MIN_QUERIES_FOR_KL = 3 # mi
MIN_QUERIES_FOR_RATE = 13 #
MIN_DURATION_FOR_RATE = 0.1
MIN_CHARS_FOR_ENTROPY = 10
MIN_QUERIES_FOR_RATIO = 1 #
MIN_QUERIES_FOR_FFT = 1 # m

```

Figure 5.4.1A: Low Values for Size Guards

```

(kali@kali)-[~/dnstunnelling]
$ python3 classifier/classifier.py data/benign.pcap
===== mozilla.com =====
score=0.5216 classification=suspicious features_used=6
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
char_kl_divergence         : raw=3.1426 suspicion=1.0000 weight=2.5
unique_subdomain_ratio     : raw=1.0000 suspicion=1.0000 weight=2.0
shannon_entropy            : raw=3.4937 suspicion=0.0000 weight=1.5
mean_subdomain_length      : raw=25.0000 suspicion=0.5000 weight=1.5
bigram_kl_divergence        : raw=7.9483 suspicion=0.9741 weight=0.5

===== mozilla.org =====
score=0.2452 classification=benign features_used=6
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
char_kl_divergence         : raw=1.8459 suspicion=0.2306 weight=2.5
unique_subdomain_ratio     : raw=1.0000 suspicion=1.0000 weight=2.0
shannon_entropy            : raw=2.8074 suspicion=0.0000 weight=1.5
mean_subdomain_length      : raw=7.0000 suspicion=0.0000 weight=1.5
bigram_kl_divergence        : raw=6.4838 suspicion=0.2419 weight=0.5

```

Figure 5.4.1B: False Positive Suspicious Value for Benign Data

Conversely, setting these too high led to an exorbitant number of requests being classified with the “insufficient_data” label, which itself is not useful.

```

SMOOTHING = 1e-10 # small co
MIN_QUERIES_FOR_KL = 10 # mi
MIN_QUERIES_FOR_RATE = 10 #
MIN_DURATION_FOR_RATE = 1.0
MIN_CHARS_FOR_ENTROPY = 50 #
MIN_QUERIES_FOR_RATIO = 10 #
MIN_QUERIES_FOR_FFT = 10 # m

```

Figure 5.4.1C: High Values for Size Guards

```

(kali@kali)-[~/dnstunnelling]
$ python3 classifier/classifier.py data/benign.pcap
===== mozilla.com =====
score=N/A classification=insufficient_data features_used=2
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
mean_subdomain_length      : raw=25.0000 suspicion=0.5000 weight=1.5

===== mozilla.org =====
score=N/A classification=insufficient_data features_used=2
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
mean_subdomain_length      : raw=7.0000 suspicion=0.0000 weight=1.5

===== githubassets.com =====
score=N/A classification=insufficient_data features_used=2
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
mean_subdomain_length      : raw=6.0000 suspicion=0.0000 weight=1.5

```

Figure 5.4.1D: Insufficient Data Results with Exorbitantly High Sample Guards

```

SMOOTHING = 1e-10 # small co
MIN_QUERIES_FOR_KL = 5 # min
MIN_QUERIES_FOR_RATE = 8 # m
MIN_DURATION_FOR_RATE = 1.0
MIN_CHARS_FOR_ENTROPY = 30 #
MIN_QUERIES_FOR_RATIO = 3 #
MIN_QUERIES_FOR_FFT = 3 # mi

```

Figure 5.4.1E: Middle-Ground Values for Size Guards

```

(kali@kali)-[~/dnstunnelling]
$ python3 classifier/classifier.py data/benign.pcap
===== mozilla.org =====
score=N/A classification=insufficient data features_used=2
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
mean_subdomain_length      : raw=7.0000 suspicion=0.0000 weight=1.5

===== githubassets.com =====
score=0.1114 classification=benign features_used=7
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
char_kl_divergence         : raw=2.2668 suspicion=0.5112 weight=2.5
unique_subdomain_ratio     : raw=0.2000 suspicion=0.0000 weight=2.0
shannon_entropy            : raw=2.5850 suspicion=0.0000 weight=1.5
mean_subdomain_length      : raw=6.0000 suspicion=0.0000 weight=1.5
bigram_kl_divergence       : raw=6.4582 suspicion=0.2291 weight=0.5
peak_to_median_ratio       : raw=1.0000 suspicion=0.0000 weight=1.5

===== githubusercontent.com =====
score=0.0000 classification=benign features_used=4
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0

```

Figure 5.4.1F: More benign results

While even with a middle-ground that classifies tunnels without false negatives, a few individual domains are still likely to be classed as insufficient data; this is acceptable since a human viewer can still make a sufficient verdict based on the other traffic being classed as benign, and the alternative is an exorbitantly high false-positive rate, or even some false-negatives slipping through.

5.4.2 Normalisation Range

Initially, the normalisation range for KL divergence was too high; the range values for all normalisation range categories were set after viewing what kind of numbers the benign and tunnelling tools produce. However, Iodine produces an exorbitantly high KL divergence value, which initially led to dns2tcp being classified as suspicious, rather than tunnel.

```

"char_kl_divergence": {"low": 1.5, "high": 5.0, "weight": 2.5},

```

Figure 5.4.2A: High KL Divergence Normalisation Range


```
(kali@kali)-[~/dnstunnelling]
$ python3 classifier/classifier.py data/dns2tcp_tunnel.pcap

===== tunnel.test =====
score=0.6506 classification=suspicious features_used=8
uncommon_record_proportion : raw=1.0000 suspicion=1.0000 weight=3.0
char_kl_divergence : raw=2.2248 suspicion=0.2071 weight=2.5
unique_subdomain_ratio : raw=0.9992 suspicion=1.0000 weight=2.0
shannon_entropy : raw=5.3537 suspicion=0.7415 weight=1.5
mean_subdomain_length : raw=36.0000 suspicion=1.0000 weight=1.5
query_rate : raw=2.9916 suspicion=0.4979 weight=1.0
bigram_kl_divergence : raw=6.6221 suspicion=0.3110 weight=0.5
peak_to_median_ratio : raw=1.4168 suspicion=0.0000 weight=1.5
```

Figure 5.4.2B: False-Negative for dns2tcp

However, as discussed in §4.7, setting this lower does not cause further issues; exorbitantly high values are treated the same as high values.

5.4.3 Minimum Features Required

As discussed in §5.4.1, there was an exorbitantly high number of domains being classified as “insufficient_data”, which was partially resolved by tuning the size guards for individual features. Following this, however, the minimum number of features for a verdict was still too high; initially it was set to an arbitrary 3 value, but lowering it by one mostly fixed that issue.

```
#Min number of features required for a verdict
"min_features_for_class": 2,
```

Figure 5.4.3A: Lower Minimum Features Threshold

```
(kali@kali)-[~/dnstunnelling]
$ python3 classifier/classifier.py data/benign.pcap

===== mozilla.com =====
score=0.1667 classification=benign features_used=2
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
mean_subdomain_length : raw=25.0000 suspicion=0.5000 weight=1.5

===== mozilla.org =====
score=0.0000 classification=benign features_used=2
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
mean_subdomain_length : raw=7.0000 suspicion=0.0000 weight=1.5

===== githubassets.com =====
score=0.1114 classification=benign features_used=7
uncommon_record_proportion : raw=0.0000 suspicion=0.0000 weight=3.0
char_kl_divergence : raw=2.2668 suspicion=0.5112 weight=2.5
```

Figure 5.4.3B: Correct Benign Classification with Less Data

6 Evaluation

Following the parameter tuning on the baseline static PCAP files ([§5](#)), the test data was expanded to include a wider variety of cases, as well as the integration of DNSCat2, as a tertiary tool with considerably different behaviour. The additional data integrated is as follows:

Benign traffic data including:

- Extended standard browsing behaviour; visiting sites, search engine queries, following links through pages, etc.
- Extended periods of streaming, inducing heavy Content Delivery Network (CDN) traffic.
- Visits to GitHub, StackOverflow, and Python Repositories, where content is packaged and downloaded over the network.
- An extended period of idle activity where messaging sites, email applications, paused streaming services, etc. were all left to idle so as to produce sporadic DNS.

Iodine data of the form:

- Idle NULL records ran for an extended period.
- Idle TXT records ran for an extended period.
- Active data transfer by running `ping -c 200 -i 0.5 10.0.0.1` over the active tunnel.
- Idle NULL records on an alternative domain

DNS2TCP captures showing:

- An extended light load on DNS, generated with the command `while true; do echo "test" > /dev/tcp/127.0.0.1/2222; sleep 2; done` over an active tunnel.
- A heavy load of DNS traffic, generated with `while true; do echo "data" > /dev/tcp/127.0.0.1/2222; done` – notably, the same command as above, but without any sleep parameter to force constant traffic.
- A light load on DNS but on an alternative domain
- Sustained transfer of larger chunks of data, generated with:
`for i in $(seq 1 100); do echo "lorem ipsum dolor sit amet consectetur adipiscing elit $i" > /dev/tcp/127.0.0.1/2222 sleep 0.5 done`

DNSCat2 additions:

- An extended idle session, where the only traffic updates are keepalive requests over the tunnel.
- Running the following commands over an active shell on DNSCat2:
 - `ls -la`
 - `whoami`
 - `ps aux`
 - `cat /etc/hostname`
 - `uname -a echo "exfiltrating data" > /tmp/test`
 - `cat /tmp/test`
 - `ls /etc | head -20`
- Simulated file exfiltration over DNSCat2 using:
 - `dd if=/dev/urandom of=/tmp/exfil.bin bs=1024 count=20 2>/dev/null`
 - `base64 /tmp/exfil.bin`
- Idle session over an alternative domain

Per the evaluation script in §4.9, the tool achieved perfect scores across precision, recall and F1 for all tools and domains:

```
(kali@kali)-[~/dnstunnelling]
$ python3 evaluate.py
=== Evaluation: DNS Tunnel Detector ===

PCAP breakdown
pcap          domains  TP  FP  TN  FN
iodine_tunnel.pcap      1    1  0   0   0
dns2tcp_tunnel.pcap     1    1  0   0   0
benign.pcap            33    0  0  33   0
browsing.pcap          40    0  0  40   0
streaming.pcap         41    0  0  41   0
development.pcap       48    0  0  48   0
mixed.pcap             57    0  0  57   0
null_idle.pcap         1    1  0   0   0
txt_idle.pcap          1    1  0   0   0
null_active.pcap       1    1  0   0   0
alt_domain.pcap        1    1  0   0   0
default_light.pcap     1    1  0   0   0
default_heavy.pcap     1    1  0   0   0
alt_domain.pcap        1    1  0   0   0
sustained.pcap         1    1  0   0   0
idle.pcap              1    1  0   0   0
interactive.pcap       1    1  0   0   0
exfil.pcap             1    1  0   0   0
alt_domain.pcap        1    1  0   0   0

Confusion matrix
                Pred. tunnel  Pred. benign
Actually tunnel             14             0
Actually benign             0             219

Metrics
Precision: 1.000
Recall:    1.000
F1 score:  1.000

No misclassifications!
```

Figure 6.1: Results

7 Discussion

7.1 Implementation

It should be noted that the implementation aggregates by parent domain, not by query, meaning that a single query, even if it is long and high entropy, is almost indistinguishable from background noise traffic. This flattening is what makes the chosen statistical model valid; i.e., the features gathered (KL divergence, entropy, peak:median ratio, etc.) only become meaningful because they exist as measures of a series (Wang et al., 2021)^[4], and as such, the

tool and current implementation is not fit for isolating singular queries as part of a tunnelled communication, only a series to a parent domain.

As touched on in §4.5, features that have a zero value are treated as None. The statistical impact of this is that if missing features were a numerical zero value, the weighted average would skew towards benign; including true zero values would drag the weighted average down during the post-normalisation calculation.

Furthermore, the final evaluation (§6) flattens “suspicious” and “tunnel” into a singular class; this because the cutoff between the two thresholds (0.7 vs 0.4) is relatively small, and functionally speaking, the event of a “suspicious” classification would still demand triage in the same manner as a “tunnel”; this caveat is important so as to avoid conflation of a question of threshold tuning (§5) with a question of detection robustness (§6).

Certain features, *total_volume*, *stddev_subdomain_length*, and *unique_subdomains* are computed but ultimately absent from the final classifier calculations; it should be noted that these features are maintained and logged for the purposes of explainability and triage, per the proposed problem statement in §1.2. This is because those particular features overlap with other features which are explicitly included in the final calculations, namely *unique_subdomain_ratio* effectively representing the length and count of subdomains combined, and the fact that *total_volume* would skew results on domains with purely heavy traffic, which is not indicative of a tunnel (Wang et al., 2021)^[1].

Lastly, the inclusion of Latin-1 as the primary encoding method to avoid *UnicodeDecodeErrors* does provide an inadvertent aid in detection; any bytes that must be decoded are binned collectively into the “other” alphabet bucket, essentially making KL divergence on Iodine massively inflated and thus trivial to distinguish.

7.2 Tools

As briefly discussed in §2.3, the tools differ in mechanism, and the mechanical differences between these tools each have statistically significant implications how results should be interpreted.

Firstly, Iodine essentially establishes a full IP stack over DNS and emits a persistent stream of raw bytes which default to NULL records (code.kryo.se, n.d.)^[28]; this inflates KL divergence to extreme values for the same reasons discussed above (§7.1) across both single-character and bigram divergence. Additionally, NULL is not a record type that a normal client would emit (RBSEC, 2025)^[29], and so *uncommon_record_proportion* is inflated, essentially making Iodine extremely easy to detect, since nearly every feature of its mechanism inflates scores and flags it.

Conversely, dns2tcp operates as a TCP stack through TXT records (Debian.org, 2025)^[30]; this consistently produces longer, higher entropy subdomains, but with far lower KL divergence than Iodine. However, it should be noted that the peak:median ratio for FFT is exceptionally high because of the way that dns2tcp data was generated; the fixed *sleep* intervals make timings extremely consistent. This certainly inflates the overall weighted sum, and thus it should be noted that actual usage of dns2tcp may have a less extreme FFT periodicity value (Kim et al., 2016)^[6].

Lastly, DNScat2 uses a hex encoding, i.e., a base-16 alphabet, and thus Shannon entropy comes out far lower value, one notably similar to natural language, and well below the other

two. Additionally, it varies between TXT/MX/CNAME records rather than just TXT or NULL, so the peak:median ratio for FFT fails to capture the differing records as effectively as if it were a single record type (iagox86, 2025)^[31]. As such, detection purely via entropy or beaconing would likely lead to a false negative, ultimately lending credence to the necessity for weighting across multiple features (Born and Gustafson, 2010; Kim et al., 2016)^{[5][6]}.

7.3 Results

The perfect scores across precision, recall and F1 should be read with some stipulation:

The purpose of including idle, active and alternate domains (per §6) was to demonstrate that the detection is not dependent on specific strings or operating modes; further demonstrated by the fact that threshold and model tuning performed during §5 was extraneous to the inclusion of DNScat2. However, it should still be noted that all traffic was generated in a controlled environment with all tools running standard configurations, and thus the model tuning is based solely on that set. Sommer and Paxson, 2010^[32] demonstrate that evaluations of network detection in controlled labs tend to inflate such evaluative scores because genuine traffic is much noisier and more heterogeneous.

8 Conclusion

The proposed classification pipeline (§3.1) detected tunnels across three mechanically distinct tools using weights across features without machine learning or sanitised training data, i.e., the gap that motivated the project (§1.1), at least marginally narrowing the gap posed by the problem statement (§1.2), given the stipulations discussed in §7.3 about results.

Each tool was primarily classified by virtue of a different feature: Iodine by KL divergence, dns2tcp by subdomain length, and DNScat2 by proportion of record types. Thus, the value of the proposed model exists in how it weighs disjoint features, rather than how it aggregates families of features (e.g. entropy, record type, timing, etc.) within their isolated, respective domains.

The philosophy of explainability in classification paid dividends during development (§5), thus providing more of an argument for transparency in classifiers beyond just triage after classification.

Additionally, the perfect F1 score should not be read as an actual measure of perfect accuracy for the reasons stated in §7.3; it is more representative of internal accuracy of model tuning and thresholds, and the value of this work is in the model architecture and statistical methods, rather than the overall metric.

The primary gaps in this model are as follows: all three tools were tested with mostly default behaviours to produce traffic; a genuine adversary could likely discover FFT thresholds, restrict record types, and manipulate subdomain lengths all within benign ranges. Thus, a next step would be evaluation against more intelligent tunnels (Pierazzi et al., 2020)^[33]. Secondly, the TLD splitting does not correctly account for TLDs with multiple parts (e.g. .com vs .co.uk), and thus, future work could correctly split TLDs in this manner.

In conclusion, the detector demonstrates that there is less of a required sacrifice for either performance or explainability in a model that attempts both than existing literature may imply (Das et al., 2017; Liu et al., 2017)^{[3][13]}. Whilst considering the limitations and stipulations on results above, this work shows that there is less of a gap than is commonly assumed between

the performance of static statistical methods and machine learning approaches whilst maintaining every property the latter is criticised for lacking.

References

- [1] Wang et al. (2021) - A comprehensive survey on DNS tunnel detection, *Computer Networks* Vol. 197, p. 108322. DOI: <https://doi.org/10.1016/j.comnet.2021.108322>
- [2] Salat, Davis and Khan (2023) - DNS Tunnelling, Exfiltration and Detection over Cloud Environments, *Sensors* 23(5), 2760. DOI: <https://doi.org/10.3390/s23052760>
- [3] Das, A., Shen, M.-Y., Shashanka, M. and Wang, J. (2017). Detection of Exfiltration and Tunneling over DNS. 2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA). doi: <https://doi.org/10.1109/icmla.2017.00-71>.
- [4] Shannon, C.E. and Weaver, W. (1949). A Mathematical Theory of Communication. *Bell System Technical Journal*, [online] 27(4), pp.623–656. doi: <https://doi.org/10.1002/j.1538-7305.1948.tb00917.x>.
- [5] Born, K. and Gustafson, D. (2010). Detecting DNS Tunnels Using Character Frequency Analysis. [online] arXiv.org. Available at: <https://arxiv.org/abs/1004.4358>.
- [6] Kim, K.-H., Jang, J., Marc Ph. Stoecklin, Wang, T., Douglas Lee Schales, Kirat Dhilung Hang and Rao, J.R. (2016). BAYWATCH: Robust Beacons Detection to Identify Infected Hosts in Large-Scale Enterprise Networks. doi: <https://doi.org/10.1109/dsn.2016.50>.
- [7] Aiello, M., Merlo, A. and Papaleo, G. (2012). Performance assessment and analysis of DNS tunneling tools. *Logic Journal of IGPL*, 21(4), pp.592–602. doi: <https://doi.org/10.1093/jigpal/jzs029>.
- [8] Docker Documentation. (2024). Networking overview. [online] Available at: <https://docs.docker.com/engine/network/>.
- [9] Biondi (2026). Welcome to Scapy's documentation! — Scapy 2.4.3. documentation. [online] Available at: <https://scapy.readthedocs.io/en/latest/>.
- [10] Le Pochat, V., Van Goethem, T., Tajalizadehkhoob, S., Korczynski, M. and Joosen, W. (2019). Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation. *Proceedings 2019 Network and Distributed System Security Symposium*. [online] doi: <https://doi.org/10.14722/ndss.2019.23386>.
- [11] Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M.H., Brett, M., Haldane, A., del Río, J.F., Wiebe, M., Peterson, P. and Gérard-Marchant, P. (2020). Array Programming with NumPy. *Nature*, [online] 585(7825), pp.357–362. doi: <https://doi.org/10.1038/s41586-020-2649-2>.
- [12] Capuano, N., Fenza, G., Loia, V. and Stanzione, C. (2022). Explainable Artificial Intelligence in Cybersecurity: A Survey. *IEEE Access*, 10, pp.1–1. doi: <https://doi.org/10.1109/access.2022.3204171>.
- [13] Liu, J., Li, S., Zhang, Y., Xiao, J., Chang, P. and Peng, C. (2017). Detecting DNS Tunnel through Binary-Classification Based on Behavior Features. 2017 IEEE Trustcom/BigDataSE/ICeSS. doi: <https://doi.org/10.1109/trustcom/bigdatase/icess.2017.256>.

- [14] Docker Documentation. (2024). Compose file reference. [online] Available at: <https://docs.docker.com/reference/compose-file/>.
- [15] Docker Documentation. (2024). Networking overview. [online] Available at: <https://docs.docker.com/engine/network/>.
- [16] man7.org. (n.d.). capabilities(7) - Linux manual page. [online] Available at: <https://man7.org/linux/man-pages/man7/capabilities.7.html>.
- [17] Mockapetris, P.V. (1987). Domain names - implementation and specification. doi: <https://doi.org/10.17487/rfc1035>.
- [18] www.iana.org. (n.d.). Domain Name System (DNS) Parameters. [online] Available at: <https://www.iana.org/assignments/dns-parameters/dns-parameters.xhtml>.
- [19] docs.python.org. (n.d.). codecs — Codec registry and base classes — Python 3.11.1 documentation. [online] Available at: <https://docs.python.org/3/library/codecs.html#standard-encodings>.
- [20] Python Software Foundation (2023). json — JSON encoder and decoder — Python 3.8.3rc1 documentation. [online] docs.python.org. Available at: <https://docs.python.org/3/library/json.html>.
- [21] Python Software Foundation (2020). csv — CSV File Reading and Writing — Python 3.8.1 documentation. [online] Python.org. Available at: <https://docs.python.org/3/library/csv.html>.
- [22] Darscheid, P., Guthke, A. and Ehret, U. (2018). A Maximum-Entropy Method to Estimate Discrete Distributions from Samples Ensuring Nonzero Probabilities. *Entropy*, 20(8), p.601. doi: <https://doi.org/10.3390/e20080601>.
- [23] numpy.org. (n.d.). numpy.histogram — NumPy v1.22 Manual. [online] Available at: <https://numpy.org/doc/stable/reference/generated/numpy.histogram.html>.
- [24] numpy.org. (n.d.). numpy.fft.rfft — NumPy v1.22 Manual. [online] Available at: <https://numpy.org/doc/stable/reference/generated/numpy.fft.rfft.html>.
- [25] Sorensen, H., Jones, D., Heideman, M. and Burrus, C. (1987). Real-valued fast Fourier transform algorithms. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 35(6), pp.849–863. doi: <https://doi.org/10.1109/tassp.1987.1165220>.
- [26] Python-guide.org. (2024). Common Gotchas — The Hitchhiker’s Guide to Python. [online] Available at: <https://docs.python-guide.org/writing/gotchas/#mutable-default-arguments>.
- [27] Mccanne, S. and Jacobson, V. (1992). The BSD Packet Filter: A New Architecture for User-level Packet Capture. [online] Available at: <https://www.tcpdump.org/papers/bpf-usenix93.pdf>.
- [28] code.kryo.se. (n.d.). kryo.se: iodine (IP-over-DNS, IPv4 over DNS tunnel). [online] Available at: <https://code.kryo.se/iodine/>.
- [29] RBSEC. (2025). DNS NULL Records: The Hidden Data Transfer Method. [online] Available at: <https://rbsec.blog/articles/red-teaming/dns-null-record/>.
- [30] Debian.org. (2025). dns2tcpc(1) — dns2tcp — Debian testing — Debian Manpages. [online] Available at: <https://manpages.debian.org/testing/dns2tcp/dns2tcpc.1.en.html>.

[31] iagox86 (2025). *dnscat2/doc/protocol.md at master · iagox86/dnscat2*. [online] GitHub. Available at: <https://github.com/iagox86/dnscat2/blob/master/doc/protocol.md>.

[32] Sommer, R. and Paxson, V. (2010). *Outside the Closed World: On Using Machine Learning for Network Intrusion Detection*. 2010 IEEE Symposium on Security and Privacy, pp.305–316. doi: <https://doi.org/10.1109/SP.2010.25>

[33] Pierazzi, F., Pendlebury, F., Cortellazzi, J. and Cavallaro, L. (2020). *Intriguing Properties of Adversarial ML Attacks in the Problem Space*. 2020 IEEE Symposium on Security and Privacy, pp.1308–1325. doi: <https://doi.org/10.1109/SP40000.2020.00073>

Appendices

Appendix A: Requirements Breakdown

A.1 Functional Requirements

Requirement ID.	Description	Priority
FR-01	Parse DNS queries from PCAP files	Must
FR-02	Capture live DNS traffic from specified interface	Must
FR-03	Extract fields from queries	Must
FR-04	Aggregate queries	Must
FR-05	Calculate entropy of subdomain characters	Must
FR-06	Calculate Kullback-Leibler divergence	Must
FR-07	Calculate N-gram frequency distribution	Must
FR-08	Calculate mean and standard deviation of subdomain length	Must
FR-09	Track query volume and rate	Must
FR-10	Count unique subdomains	Must
FR-11	Track proportion of uncommon record types	Must
FR-12	Apply FFT to timing intervals	Must
FR-13	Flag domains with high frequency spikes	Must
FR-14	Calculate a “suspicion score”	Must
FR-15	Classify domains as benign or probable tunnel	Must
FR-16	Fixed time window for feature extraction	Should
FR-17	Periodic FFT recalculation during live capture as data accumulates	Should
FR-18	Configurable detection thresholds through CLI	Should
FR-19	Logging of all flagged events with timestamps	Should
FR-20	Exportable results	Could
FR-21	Support for CSV and other filetypes than PCAP	Could
FR-22	Active network interception	Won't
FR-23	Detection of alternate DNS abuse	Won't
FR-24	Encryption analysis	Won't

A.2 Non-Functional Requirements

Requirement ID.	Description	Priority
NFR-01	Generate test data using three tunnelling tools	Must
NFR-02	Evaluate detection using valid metrics (precision, recall, F1)	Must
NFR-03	Performance breakdown per tunnelling tool	Should
NFR-04	False positive analysis against noisy benign traffic	Should
NFR-05	GUI beyond CLI	Won't